

PRACTICAL-2

Task 1: Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user Space and kernel space during execution

```
ruhi0404@Ruhitha:~/Operating_Systems$ nano prac2task1.c
ruhi0404@Ruhitha:~/Operating_Systems$ echo "This is the content of the source file." > File1.txt
ruhi0404@Ruhitha:~/Operating_Systems$ cat File1.txt
This is the content of the source file.
ruhi0404@Ruhitha:~/Operating_Systems$ gcc prac2task1.c -o prac2task1
ruhi0404@Ruhitha:~/Operating_Systems$ ./prac2task1
File copied successfully.
ruhi0404@Ruhitha:~/Operating_Systems$ v
v: command not found
ruhi0404@Ruhitha:~/Operating_Systems$ cat File2.txt
This is the content of the source file.
ruhi0404@Ruhitha:~/Operating_Systems$ |
```

Observation

The program successfully copies the contents of one file to another using the `open()`, `read()`, `write()`, and `close()` system calls. During execution, the program runs in user space and requests file operations through system calls. Each system call causes a controlled transition to kernel space, where the Linux kernel performs the required file-system operation. After completing the operation, control returns to user space. Thus, the experiment demonstrates how system calls provide a safe interface between user applications and kernel-managed hardware resources.

Task 2: Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved


```

ruhi0404@Ruhitha: ~/Operati
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_ADDRESS", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_NAME", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_NAME", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_PAPER", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_PAPER", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_MESSAGES", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_MESSAGES", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_MESSAGES/SYS_LC_MESSAGES", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_MONETARY", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_MONETARY", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_COLLATE", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_COLLATE", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_TIME", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_TIME", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_NUMERIC", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_NUMERIC", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_CTYPE", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_CTYPE", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, "sample.txt", O_RDONLY) = 3
read(3, "Hello, this is a sample file for...", 131072) = 41
write(1, "Hello, this is a sample file for...", 41Hello, this is a sample file for strace.
) = 41
read(3, "", 131072) = 0
close(3) = 0
close(1) = 0
close(2) = 0

```

Observation

The strace utility was successfully used to trace the system calls generated by the command `cat sample.txt`. The system calls such as `openat()`, `read()`, `write()`, and `close()` were observed. `openat()` opens the file, `read()` reads the contents of `sample.txt`, `write()` displays the contents on the terminal, and `close()` closes the file descriptor. The experiment demonstrates how the `cat` command interacts with kernel services through system calls.